Universidad Autónoma de Madrid

Escuela Politécnica Superior

24|25

Bachelor thesis

Network malware detection using machine learning techniques based on statistics and traffic fingerprints

Eduardo Polo Peyres

Escuela Politécnica Superior
Universidad Autónoma de Madrid
C\Francisco Tomás y Valiente nº 11

Campus Internacional excelencia UAM CSIC+

www.uam.es

UNIVERSIDAD AUTÓNOMA DE MADRID
ESCUELA POLITÉCNICA SUPERIOR

Bachelor as Ingeniería Informática Bilingüe

BACHELOR THESIS

Network malware detection using machine learning techniques based on statistics and traffic fingerprints

Author: Eduardo Polo Peyres
Advisor: Jorge Enrique López de Vergara Méndez

mayo 2025

Eduardo Polo Peyres
Network malware detection using machine learning techniques based on statistics and traffic fingerprints

Eduardo Polo Peyres
C\ Francisco Tomás y Valiente Nº 11

PRINTED IN SPAIN

Resumen

El mundo de la comunicación en red y la transmisión de datos depende en gran medida de los protocolos de cifrado, lo que ha hecho que el *malware* también evolucione en su uso del tráfico cifrado. Si bien las tecnologías de cifrado como *TLS (Transport Layer Security)* son esenciales para proteger la privacidad del usuario y la confidencialidad de los datos, también ayudan a ocultar las actividades maliciosas, haciendo que los métodos de detección tradicionales sean menos efectivos. Según los informes [1], el 71 % del *malware* analizado utiliza protocolos de cifrado como *SSL (Secure Sockets Layer)*.

El objetivo de este Trabajo Fin de Grado es explorar la aplicación de técnicas de aprendizaje automático para identificar la actividad maliciosa sobre el tráfico cifrado. Para lograr este objetivo, el trabajo examina varios métodos de clasificación de *malware* mediante el análisis de características específicas del tráfico *TLS*. En particular, analiza la aplicación de las huellas *JA4+*, una colección de técnicas de huellas *TLS*, junto con el cálculo de estadísticas de los datos de flujo de red, inspirándose en el artículo de *MalDIST* [2], para identificar y categorizar el tráfico de *malware*.

Las huellas *TLS* presentan una forma útil y eficaz para la detección de *malware* al analizar las características específicas del *handshake TLS*, lo que permite la identificación de actividad y posibles actores maliciosos. Además, las características estadísticas derivadas de los flujos de red pueden proporcionar información adicional y complementaria para mejorar la precisión de la detección.

Finalmente, este trabajo busca evaluar la eficacia de la combinación de las huellas *JA4+* con el análisis de estadísticas de flujos de red para la detección de *malware* en el tráfico cifrado. Al adaptar y ampliar el modelo *MalDIST*, este estudio busca contribuir al desarrollo de metodologías de detección de malware más robustas y precisas en un entorno de red cada vez más cifrado.

Palabras clave

TLS, *SSL*, *JA4+*, *MalDIST*, *malware*

Abstract

The whole world of network communication and data transmission relies heavily on encryption protocols, which has made malware evolve its use of encrypted traffic as well. While encryption technologies such as *TLS (Transport Layer Security)* are essential for protecting user privacy and data confidentiality, they also obscure malicious activities, making traditional detection methods less effective. According to reports, 71 % of the malware covered uses encryption protocols such as *SSL (Secure Sockets Layer)* [1].

The aim of this bachelor thesis is to explore the application of machine learning techniques to identify malicious activity on encrypted traffic. In order to achieve this goal, this work examines several methods for classifying malware by analyzing specific characteristics of *TLS* traffic. In particular, it looks at the application of *JA4+* fingerprints—a collection of *TLS* fingerprinting techniques—alongside statistical evaluations of network flow data, taking inspiration from the *MalDIST* article [2], to effectively identify and categorize malware traffic.

TLS fingerprinting presents a valuable opportunity for malware detection by analyzing the unique characteristics of the *TLS* handshake, allowing for identification of applications and potentially malicious actors. In addition, statistical attributes derived from network flows can provide supplementary differentiating information to improve detection precision.

Finally, this work seeks to evaluate the efficacy of combining *JA4+* fingerprints with statistical newtork flow analysis for malware detection in encrypted traffic. By adapting and extending the *MalDIST* model, this study seeks to contribute to the development of more robust and accurate malware detection methodologies in an increasingly encrypted network environment.

Keywords

TLS, *SSL*, *JA4+*, *MalDIST*, malware

Table of Contents

Lists

List of algorithms

List of equations

List of figures

List of tables

1

Introduction

This chapter will cover the realization of the project with its motivation, objectives, procedural phases and the document structure.

1.1. Motivation

The continuous evolution of network traffic presents an ongoing challenge for malware detection systems, requiring continuous updates and adaptation. In addition, malicious actors are constantly exploiting new techniques to bypass intrusion detection systems using encrypted channels, such as TLS tunnels, to hide commands and control communications, updates and data extraction, creating an urgent need for innovative approaches to accurately identify malware from benign traffic.

Reports ensure that the majority of malware attacks come from encrypted traffic (71 % of malware installed from phising is hiding in encryption [1] and 85.9 % of threats are delivered over encrypted channels [3]). This work is motivated by the critical need to improve malware detection capabilities (both in terms of accuracy and speed) to solve this growing challenge.

The exploration of TLS fingerprints offers a promising and innovative way for identifying malware based on some accessible characteristics available at the beginning of a connection and the statistics of the traffic flow. Furthermore, the development of machine learning techniques has the potential to provide more reliable results and future predictions.

Therefore, this study is motivated by the aim of contributing to the advancement of malware detection methodologies through the examination of JA4+ fingerprints in conjunction with statistical flow analysis using deep learning. The results of this research are expected to contribute to the development of more effective security measures to protect works and systems against evolving malware threats.

1.2. Objectives

The main goal of this bachelor thesis is the implementation of a machine learning model able to detect if a traffic capture contains malware or just benign traffic. To do this, the following objectives have to be met:

- Discover how traffic is presented through the network and how malware behaves.
- Study the previous techniques to discover malware through the network.
- Build up a database of malware families pcap files and benign application pcap files from Malware-analysis.net, University of new Brunswick-Canada and Tria.ge Sandbox.
- Create a model that detects malware based on statistical features of the packets of a flow and another model that detects malware analyzing the *JA4+* fingerprints based on two articles abroad in section 2.5.
- Implement a hybrid model with those 2 models.
- Evaluate the results, efficiency and accuracy of the hybrid model in comparison with each model isolated.

1.3. Realization Phases

This section reflects the time invested and the way in which the project was planned. It began in October 2024 and ended in May 2025, a total of 8 months. The defense is expected to take place in June 2025. To document the timeline, a Gantt chart has been created, as shown in figure 1.1.

1.4. Document Structure

The document is organised in the following way:

- **Chapter 1**: This first chapter begins the explanation of the work, encompassing the reasons and objectives for its realization. Furthermore, a Gantt chart is presented, where the time invested in each of the chapters can be seen, as well as the present structure of the rest of the document.
- **Chapter 2**: State of the Art, contextualizes and explains the origins of malware detection in encrypted traffic. It describes malware behavior within networks and the impact of the TLS protocol. Furthermore, it explains JA4+ fingerprints, including their structure and utility for malware identification. Finally, it presents a review of previous work in malware detection.

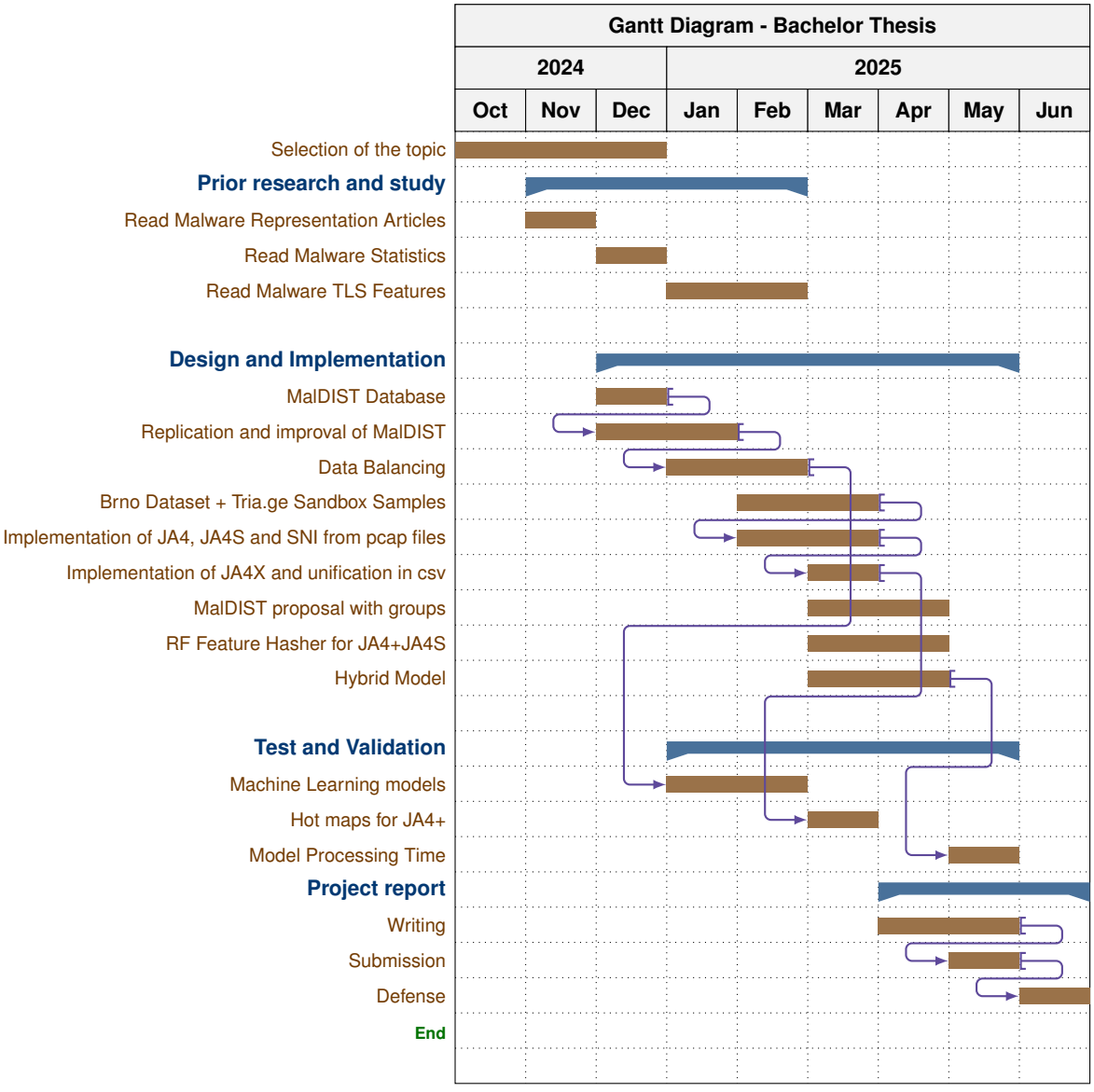


Figure 1.1: Gant Diagram.

- **Chapter 3**: The Design and Implementation chapter details all the tools employed to develop the objectives of the project. It illustrates how data is arranged and explains the implementation of the previous work models with its modifications and designs. Finally, it explains the proposed methodology combining the two models replicated, describing the realization phases.
- **Chapter 4**: This chapter presents and discusses the tests conducted and the resulting outcomes for both the replicated work and the proposed implementation. Visual representations of the data are provided through graphics and tables. The analysis in this chapter will form the basis for the conclusions drawn in the subsequent chapter.
- **Chapter 5**: This chapter is reserved for the final conclusions and the future work that can be made in order to improve or implement the uncovered topics of this project.

2

State of Art

2.1. Introduction

This chapter will explain and go through the principal concepts of the work done for this barchelor thesis. First, it will explain what Network Malware is and how to detect it, as it is the principal subject to work with and is taken as a sample to obtain results. Then, in section 2.3 the concept of the data encryption through the network with *TLS* will be explained, and section 2.4 will cover methods to work with encrypted traffic, the *JA3* and *JA4+* fingerprints. Finally, section 2.5 will go through all the articles and previous works and solutions for malware detection and classification.

2.2. Network malware

Network Malware is basically malicious software designed to damage, exploit or compromise devices, networks or data through that network. Malware can be distributed in various ways, but through the network is the most frequent one, and where malicious actors focus on, as it is where most of the users are, and it is accessible to all the world. So, firstly, this article studies how network malware is present and hidden through the net.

To do this, this article studies the PCAP (Packet CAPture) files, data files which collect all the network traffic presented as packets from a specific network connection over a certain period of time. Using *Wireshark* as the only tool, there are several methods to make a guess in which files may present malicious activity:

- Looking at the DNS packets, which ask for the user's IP lookup. This could mean an attacker is trying to know what IP the user is coming from. However, this is not sufficient evidence to prove the user is being infected, and it is not present in every malware activity.
- Looking at HTTP packets and following the TCP or HTTP stream to analyze the HTTP request, which gives much information to know if the session is trying to infect the system, like suspicious ports, hosts and executable code hidden in the payload. This can be observed by exporting HTTP

objects and analyzing them. Nevertheless, this method can be only executed on plaintext traffic, but most of the web traffic is now encrypted, which does not allow seeing this information. Malware has also adapted to encrypt their information, as mentioned in the Motivation.

- Looking at the *Client Hello* packet in the pcap file, which contains much useful information. The *TLS version*, *Cipher Suite* and *JA3* fingerprints, among other parameters, can differentiate Malware from Benign files, but this will be covered in the following sections.

Network malware has been classified over the time depending on how they attack the victim's device. While there are many different variations of Malware, Table 2.1 shows the most common types:

Type	Descritption	Example
Ransomware	Disables victim's access to data until ransom is paid	RYUK
Fileless Malware	Makes changes to files that are native to the OS	Astaroth
Spyware	Collects user activity data without their knowledge	DarkHotel
Adware	Serves unwanted advertisements	Fireball
Trojans	Disguises itself as desirable code	Emotet
Worms	Spreads through a network by replicating itself	Stuxnet
Rootkits	Gives hackers remote control of a victim's device	Zacinlo
Keyloggers	Monitors users' keystrokes	Olimpic Vision
Bots	Launches a broad flood of distributed attacks	Echobot
Mobile Malware	Infects mobile devices	Triada
Wiper Malware	Erases user data beyond recoverability	WhisperGate

Table 2.1: Types of Malware [4]

The types of malware samples taken are mostly Trojans, focused on financial theft.

2.3. The Transport Layer Security (TLS)

TLS (Transport Layer Security) and its predecessor SSL (secure Socket Layer) are protocols implemented on top of TCP that encrypt data transmitted between computers, TLS being a newer version offering better security. Their primary goal is to provide a secure channel between two communicating peers. This secure channel should provide the following properties: authentication, confidentiality and data integrity [5].

TLS/SSL starts establishing a secure connection by what is called a handshake. The security handshake begins when a client sends a *Client Hello* to the server with the information about the encryption algorithm and options that the client can support. The server receives this *Client Hello*, chooses an encryption algorithm, and responds with a *Server Hello*, which contains information about

the encryption algorithm chosen and the SSL certificate with server domain name and public key used for encryption.

This handshake process, while establishing a secure connection, also reveals valuable information about the communicating parties, like supported encryption protocols, cipher suites, and other parameters useful to create a unique 'fingerprint' of the connection. This technique, known as *TLS fingerprinting* [6], allows the identification and categorization of clients (such as web browsers, applications, or bots) based on the unique characteristics of their TLS handshake. In this work, one type of fingerprinting method, *JA4+*, is used as an approach to malware detection.

2.4. JA4+ fingerprints

JA4+ is a collection of TLS fingerprints developed by John Althouse *et al.* in 2023 [7], replacing the *JA3 TLS fingerprinting* developed previously in 2017. These methods consist of extracting specific attributes from the TLS handshake and hashing them to identify applications in encrypted traffic. In this work, these fingerprints will be used to scan for threat actors and malware detection.

2.4.1. Evolution from JA3

JA4+ was released in 2023 as an upgrade method for *JA3*, created in 2017 by a trio of Salesforce researchers: John Althouse, Jeff Atkinson and Josh Atkins, whose initials gave the name to the fingerprint. *JA3* fingerprint is divided into two different fingerprints: the client side (*JA3*) and the server side (*JA3S*).

The client side gathers the following fields in the *Client Hello* TLS record: *TLSVersion, accepted ciphers, List of Extensions, Elliptic Curves, Elliptic Curves Format* in this order. Then it concatenates each field separated by "," and delimiting by "-" each value if one field has more than one value. JA3S does the same in the *Server Hello* message with the following fields: *TLSVersion, Cipher, Extensions* in this order. Then, both strings are MD5 hashed to produce a 32-character fingerprint. These are *JA3* and *JA3S* fingerprints.

These fingerprints are useful in security scenarios for a number of reasons. *JA3* enables the identification of malicious client applications, potentially detecting malware using specific *TLS* libraries or configurations. *JA3S* can be useful in identifying the infrastructure of a server and detecting abnormalities when, for example, anomalies are found from server behavior may be a bug or an attack. By combining *JA3* and *JA3S*, a more comprehensive view of the *TLS* handshake is achieved, enhancing the ability to detect malicious activity within encrypted traffic without requiring decryption [8].

2.4.2. How does JA4+ work

JA4+ tries to do the same as its predecessor, gather some fields from the packets which establish the TLS connection, but with a different approach. To start with, *JA4+* is a set of specific network fingerprints for multiple protocols that are both human-readable and machine-readable. For all types the structure of the fingerprints is the same, *a_b_c* format, which allows separating the fingerprint and using only part of it to detect malicious activity (for example using ab or ac or c).

Table 2.2 shows the *JA4+* fingerprints released:

Full Name	Short Name	Description
JA4	JA4	TLS Client Fingerprinting
JA4Server	JA4S	TLS Server Response / Session Fingerprinting
JA4HTTP	JA4H	HTTP Client Fingerprinting
JA4Latency	JA4L	Latency Measurement / Light Distance
JA4X509	JA4X	X509 TLS Certificate Fingerprinting
JA4SSH	JA4SSH	SSH Traffic Fingerprinting
JA4TCP	JA4T	Passive TCP Client Fingerprinting
JA4TCPServer	JA4TS	Passive TCP Server Response Fingerprinting
JA4TCPScan	JA4TScan	Active TCP Server Fingerprinting

Table 2.2: *JA4+* types of fingerprints [7].

This work focuses on JA4 (client side), JA4S (server side), and JA4X (certificate).

- **JA4 – TLS Client Fingerprint**: looks the *TLS Client Hello* record and builds fingerprint. Figure 2.1 shows the format where ALPN is Application-Layer Protocol Negotiation, a *TLS* extension that allows the application layer to negotiate which protocol should be used over the secure connection, and the Cipher Suite is the set of algorithms that determine how communication of safe connection is protected, how data are encrypted and authenticated.
- **JA4S – TLS Server/Session Fingerprint**: After client sends *Client Hello*, server responds with *TLS Server Hello* record. The *Server Hello* is unique for both server app and *Client Hello*, and different *Client Hello* may cause different *Server Hello*. Figure 2.2 shows the format in this case.
- **JA4X – X509 TLS Certificate Fingerprint**: Based on the X509 TLS Certificate [9], the fingerprints indicate the way TLS certificates are generated, not the values within the certificate. This approach can identify applications and settings used to create the certificate, which can be highly valuable in threat hunting, as threat actors may utilize diverse certificates but tend to use the same methods to create said certificates. Figure 2.3 shows the format, where RDN (Relative Distinguished Names) is a set of strings that uniquely identifies an object within a directory or naming system. In the context of X.509 certificates, it uniquely identifies the certificate's subject or issuer. The issuer is

the entity that signed and issued the certificate and the subject is the entity whose public key is being certified.

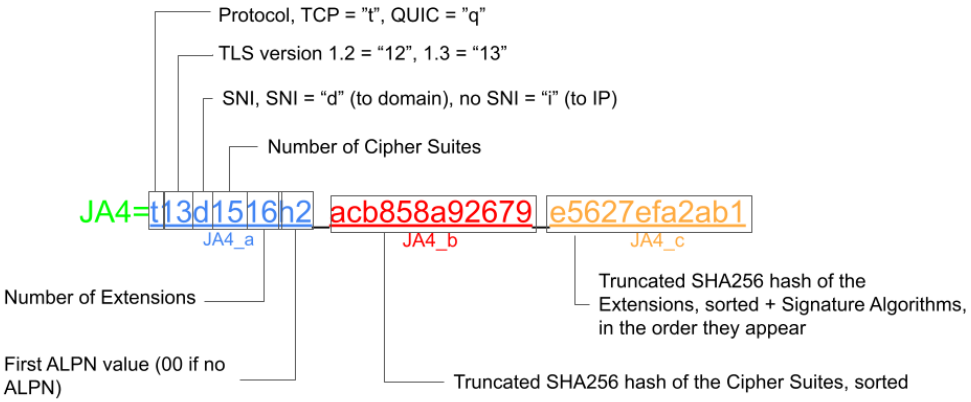


Figure 2.1: *ja4* format [7]

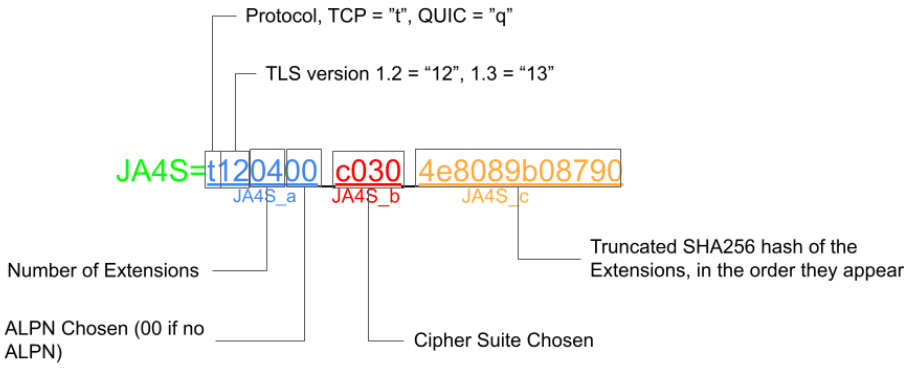


Figure 2.2: *ja4s* format [7]

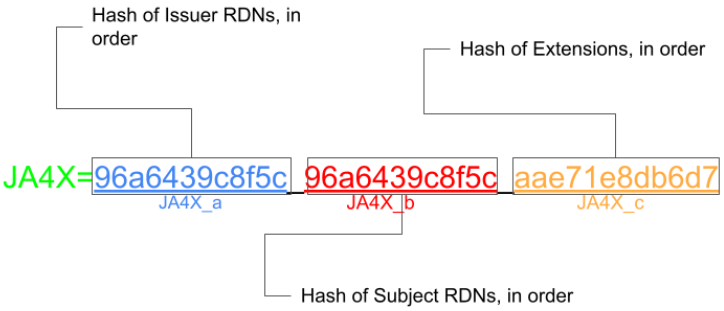


Figure 2.3: *ja4s* format [7]

2.5. Related Work

Network traffic classification is crucial for various network management and security tasks, including Quality of Service (QoS) management, intrusion detection and malware detection. Traditional methods for traffic classification have been challenged by the increasing prevalence of encrypted traffic, which disguise the payload, moving to approaches based on featured extraction from the TLS handshake, additional protocols such as HTTP and DNS, or flow statistics.

2.5.1. Malware Representation

Before malware encryption was so present in network traffic, there were a lot of proposed solutions to malware detection with a common input: malware representation into images. Nataraaj [10] proposed a transformation of a malware binary file into an image by organizing the bits in a 2D array and applying Grayscale; then he will be able to predict and differentiate 8 families of malware by applying to the images Gabor filtering and creating a model of K-NN to process these images. This approach was useful to predict if a binary file belongs to one malware family or not, but if the malware files are hidden through traffic, it becomes useless. In addition, applying Gabor filtering to implement the K-NN is high costly in comparison with CNNs.

Other papers started working on pcap files by representing malware and benign traffic through the payload of the packets. Wang [11] proposed a custom LeNet-5 (CNN) neural network architecture for Malware Detection to recognize different image-like patterns trained on the n first payload bytes of the flow (Matrix size $\sqrt{n}$, $\sqrt{n}$). Later, they also proposed the bytes view not as an image but as a sequential time series, a single array of length 784 trained with the first n payload bytes of the flow

and processed with a 1D CNN [12]. The problem is that, with encrypted traffic, the payload would be encrypted, generating noise to the CNN, which will start predicting by the size of the payload more than the content.

2.5.2. Malware Statistics

Flowpic [13] introduced a novel method for classifying encrypted traffic by representing the flow data as images, called FlowPics, by creating a matrix where the rows are the size of the flow packets and the columns the normalized arrival time, and then, using CNNs for classification. This can become a bad solution as it is easy to evade by malware authors. They just need to adapt their traffic distribution as normal traffic behavior.

DISTILLER [14] is a multimodal multitax deep learning model for traffic classification. It is divided in two different single modality neural networks, one processed with the 784 first bytes of the payload of the flow and the other processed with the following protocol fields of the first 32 packets: [size, IAT, direction, TCP window-size].

MalDIST [2] is a DISTILLER variant prepared for malware detection and classification. The architecture is practically the same, being also a multimodal model separated in a single modality neural network for the payload, a single modality neural network for the protocol fields, and it adds a (5,14) matrix of statistics of the first 32 packets of the flow. This matrix contains 5 different groups: 1. Bidirectional Packets (all the flow packets), 2. Src2Dst Packets (from SrcIP to DstIP), 3. Dst2Src Packets (from DstIP to SrcIP), 4. Handshake Packets (TCP handshake) and 5. Data Packets (non TCP handshake) where SrcIP is the IP who sends the first packet SYN and DstIP is the IP who responds with SYN-ACK. Then, inside each group of packets, the following fields are calculated: min, max, mean, standard deviation and skewness of the sizes and IAT, and additionally the total number of bytes, total number of packets, byte/s rate and packet/s rate. This approach has a 98.52 % accuracy and seems to be a good solution for Malware Detection, but it has some issues: The payload will skew the model as traffic is encrypted, the dataset used to train is unbalanced, and the model is very complex for the input parameter.

Ariel University of Israel proposed an Open-Source Framework for Encrypted Internet and Malicious Traffic Classification (OSF-EIMTC) [15], which included different models, with different feature extraction for each model and trained with a set of different datasets. Basically, this source put together the implementation of some of the previous articles seen about malware detection and classification in a common framework and studied their results.

2.5.3. Malware TLS Features

Appart from calculating statistics, which is a valid solution to extract features from encrypted traffic, other approaches focused on *TLS fingerprinting* for their models. Two researchers from *Cisco* proposed a novel *TLS fingerprinting* method [16] which incorporated, in addition to analyzing *Client Hello*, destination context: the destination IP address, port, and server name. The system employs a weighted Naïve Bayes classifier, achieving a 99.9 % of precision and 88.7 % of recall. The problems of this paper is the amount of false positives in benign traffic and adding the destination context will make the prediction more specific rather than general, meaning that the model learns not just “this TLS fingerprint is associated with malware,” but “this TLS fingerprint, when going to this specific IP/server, is likely malware.”

Researchers at University of Brno introduced studies working with *JA3* and *JA4+* fingerprints. One of their papers [17] uses *JA4+* with *SNI* for malware detection in encrypted traffic. They collected pcap files of Desktop Malware and Benign Applications using *Tria.ge* sandbox and *Wireshark* capturing and Mobile Malware and Benign Applications using *AVD Emulation*. Then, they calculated *JA4*, *JA4S*, *JA4X* and *SNI* of the pcap files of each app and combined them. Finally, they built several heat maps comparing the different malware families and the malware families with the benign apps. They concluded the best approach was combining *JA4* with *JA4S* and *SNI* with 89.3 % unique fingerprints, useful to differentiate each family of malware and most important, malware from benign apps.

2.6. Conclusions

This chapter has provided an overview of key concepts and existing research relevant to the detection of network malware in the context of encrypted traffic. It has explored the challenges presented with the encryption of traffic and how malicious activity can be hidden easily, making traditional detection methods less effective. The chapter has also detailed the evolution of *TLS fingerprinting* techniques, highlighting the potential of *JA4+* fingerprints as a powerful tool to identify and fingerprint the malicious actors. Additionally, this chapter has reviewed previous works, which help to know about the techniques already applied and the results obtained. Specifically, this bachelor thesis will focus on [2] and [17] to create a model and try to upgrade these methods for malware detection.

3

Design and Implementation

3.1. Introduction

Having established the theoretical bases of the topic and reviewed the current state of the art in network malware detection, particularly concerning with encrypted traffic and the previous work techniques employed, this chapter will cover all the practical procedures followed in design and implementation to achieve the objectives of this final project. This will include going through the programming environment and specific tools employed for the development. Subsequently, it will show the structure of the dataset and the sources from which the dataset samples were extracted. Finally, the chapter will display a model design indicating the architecture, steps taken and selection of the machine learning algorithms of both the replicated work and the model proposed, explaining them with detail.

3.2. Programming Environment

The development and testing phase of this project was carried out on an *Ubuntu 22.04* system. We chose this particular version for its proven stability, which is especially important when working with large datasets and running sophisticated machine learning processes. The user-friendly interface of *Ubuntu*, combined with its powerful command-line tools, made managing numerous PCAP and CSV files much easier during the project.

Command-line interfaces and shell scripts (.sh files) contributed considerably to automating routine tasks like file handling, data preprocessing, and running training and evaluation routines. *Wireshark* was also heavily used for detailed analysis of PCAP files, which helped to analyze the traffic captured and verify the differences between malware PCAP files and benign PCAP files.

To handle the large amount of data generated and processed, all the data files were kept on a *Maxtor* external hard drive. This external storage provided enough space to store roughly 34 GB of data, which included raw PCAP files and processed CSV files. For organizing our code, *Visual Studio Code (VS Code)* was used as the main IDE. Its features for code management, debugging, and version control helped keep our workflow clear and efficient. The project was structured into three main folders:

one for the *MalDIST* implementation, another for scripts and analyses related to the replication of the *JA4+* report, and a third for the development of hybrid models. This division clarified the two different approaches to achieve the goal of the project and helped in scalability.

3.2.1. Used Tools

- *Wireshark* used to analyze pcap files and understand network traffic and the library *tshark* from *Wireshark* to extract and handle information from the pcap files. *Tshark* is used in the command line or in shell scripts as the *Wireshark* display filter.
- Jupyter notebook to perform all the graphics and plots and to compute the machine learning models. Its organization in cells allows executing small pieces of code and show the results little by little, making this useful to divide the code in different phases (for example first manipulating data and showing it, then designing the model and showing the results).
- *NumPy* library from Python to work with arrays and *Pandas* library from Python, built on top of NumPy, to manipulate and analyze data, especially from the CSV files.
- *Scapy* library from Python to access to the PCAP files and the packet objects properties (like protocols, IP, flags) in a Python dictionary way (for example accessing the TCP layer of a packet is “packet[TCP]”).
- *Sklearn* main library from Python for Machine Learning which includes all the models tried in this project (Neural Networks, Random Forest, FeatureHasher), balancing tools used as SMOTE and metrics to test the efficiency of these models.
- *Seaborn* and *Matplotlib* Python libraries to represent results in graphics.

3.3. Dataset

The dataset created for this project consists of two directories: Dataset 1, inspired from *MalDIST* proposal [2], and Dataset 2, inspired from *JA4+* proposal [17]. The manipulation and processing of the data, necessary for the implementation of the two models, will be safe in the *Maxtor* external device and will occupy 34 GB of space as mentioned in Section 3.2. But in the end, there will be a final database, the result of the combination of Dataset 1 and Dataset 2 to compute the hybrid model. The database only contains PCAP files extracted from the corresponding sources and CSV files that contain some information from the PCAP files to be processed by the Machine Learning Models. The main characteristics of each Dataset will be explained in the following subsections:

3.3.1. Dataset 1: MalDIST DB

The statistical model will take the PCAP files from the same sources as the original paper [2], including the same malware families (*Dridex*, *Emotet*, *Hancitor* and *Valak*) from the website Malware-Analysis.net [18] and benign traffic are taken from ISCXVPN2016 database from the Canadian Institute for Cybersecurity [19]. Figure 3.1 shows the Dataset 1 sources.

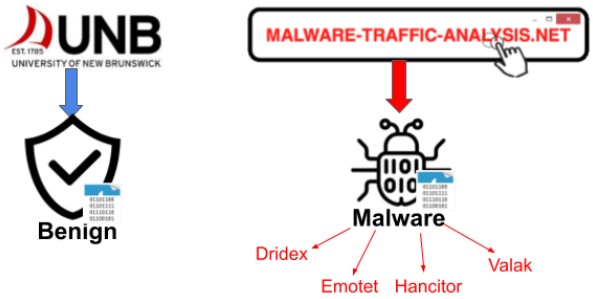


Figure 3.1: Dataset 1 PCAP files

Each PCAP file will be processed, and two different formats will be produced and saved in different directories:

- **Sessions**: *pcaps* file which contain all the sessions of the PCAP files in the MalDIST/SESSIONS directory of *Maxtor* drive.
- **Features**: *csv* file with the statistical features of each session pcap in the MalDIST/FEATURES directory of *Maxtor*.

3.3.2. Dataset 2: JA4+ DB

The fingerprint dataset will take the PCAP files from the *Brno University JA4+* report plus some benign samples resulted from executing benign desktop apps on *Tria.ge sandbox*, as there were not enough benign fingerprints in comparison with malware to compute the model. Figure 3.2 shows the Dataset 2 sources.

Each PCAP file will be processed in a CSV saved in the directory JA4, which contains the *JA4*, *JA4S*, *JA4X* and *SNI* per {SrcIP, DstIP, SrcPort, DstPort} of each file.

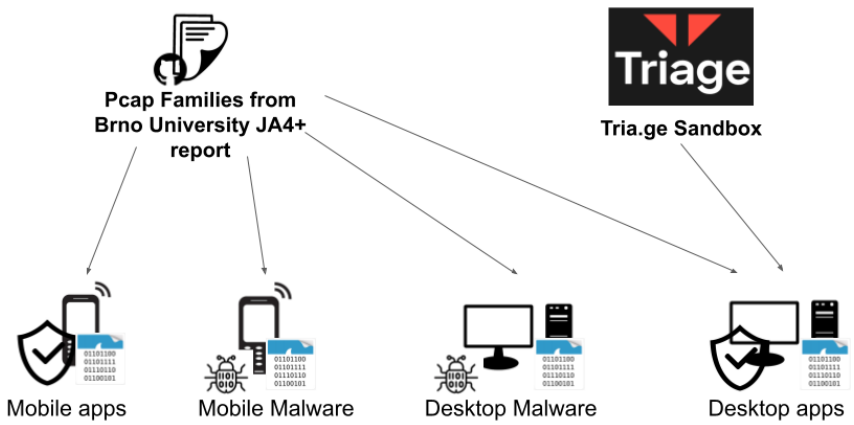


Figure 3.2: Dataset 2 PCAP files

3.3.3. Model DB

For the final hybrid model, the database labels will be reduced into desktop benign apps and desktop malware, creating a single class model with the two different classes balanced. So, for the malware label it will only take into account the malware families from the Brno paper, while for the benign apps label it will include desktop apps from Brno paper, the benign pcap files from *ISCXVPN2016* and the *Tria.ge* Sandbox pcap files to keep the whole dataset well balanced.

3.4. Improvements over Previous Work

This section will describe the steps taken and decisions made to implement the two solutions chosen for malware detection in encrypted traffic: a model based on the statistics of the network flows and a model based on the fingerprints.

3.4.1. MalDIST

The replication of *MalDIST* [2] had several modifications that will be covered through these subsections, but the principal ones are the simplification with the machine learning model selection, the preprocesing of the packets and balancing data. Figure 3.3 shows a schematic representation of the design of the MalDIST approach implemented in this project.

As depicted in Figure 3.3, the initial step to build this statistical model involves partitioning each PCAP file into individual flows. This is accomplished using the ‘tshark‘ command-line tool with the

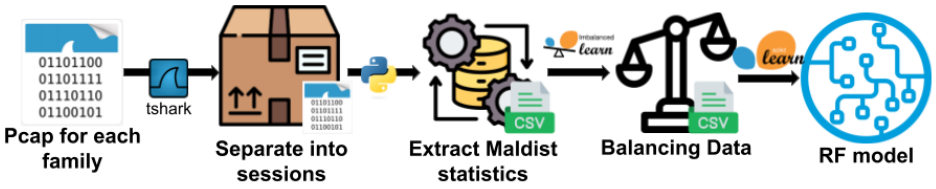


Figure 3.3: *MalDIST* design

following syntax:

```
tshark -r "$input_pcap_file" -T fields -e tcp.stream | sort -n | uniq
```

Following this, each network flow, now separated into individual PCAP files, was subjected to feature extraction. This process involved calculating various statistical metrics and extracting protocol header information, as described in Subsection 2.5.2. Importantly, there is an intentionally exclusion of the packet payload from this step. Since the dataset consists primarily of encrypted traffic, including the payload could introduce irrelevant and unpredictable data into the feature set. This may add noise to machine learning models, potentially affecting their accuracy. Because encryption conceals the content of the data, any patterns within the payload are unlikely to reliably indicate malicious activity. All the feature extraction of the pcap flows is saved into CSV files with the file name, label of the family they belong to and 198 features (5 groups with the 14 statistics + 32 packets with the protocol headers). Algorithm 3.1 describes the code to extract the features of a flow.

The 5 different labels are Dridex, Emotet, Hacintor, Valak and Benign. But the problem is that the flows generated by Hacintor, for example, are way more than the flows from the Benign apps, resulting in an unbalanced dataset which will produce poor generalization performance due to overfitting to the dominant class in the training set. To mitigate the issue, a data balancing technique, focused on data augmentation, was employed, *SMOTE* (Synthetic Minority Over-Sampling Technique), which addresses class imbalance by generating synthetic samples for the minority classes based on the features of the members of the same family. In this study, SMOTE was utilized to oversample the minority classes, managing to obtain a target class size of 7000 samples for each family and obtaining good results.

Once the dataset is balanced, several models have been tested to compare the results, being Random Forest the model with the best results, with an Accuracy of 0.9889. Other models tested were RF with Extratree Classifier, XGBoost, KNN and Logistic Regression.

Input: PCAP file $pcap\_file$
Output: Statistical matrix and protocol matrix

$packets \leftarrow$ rdpcap($pcap\_file$);
Initialize dictionary `groups` with keys: `bidirectional`, `srcdst`, `dstsrc`, `handshake`, `datatransfer`;
if $packets$ *is empty* **then**
    return $(None, None)$;
end
$(src\_ip, dst\_ip) \leftarrow$ identify_src_dst_ips($packets[:32]$);
foreach $packet$ *in* $packets[:32]$ **do**
    Add $packet$ to `groups["bidirectional"]`;
    if src_ip *and* dst_ip *are defined* **then**
        if $packet.IP.src = src\_ip$ *and* $packet.IP.dst = dst\_ip$ **then**
            Add $packet$ to `groups["srcdst"]`;
        end
        else if $packet.IP.src = dst\_ip$ *and* $packet.IP.dst = src\_ip$ **then**
            Add $packet$ to `groups["dstsrc"]`;
        end
    end
    if $packet$ *contains TCP layer* **then**
        if *SYN flag is set* **then**
            Add $packet$ to `groups["handshake"]`;
        end
        else
            Add $packet$ to `groups["datatransfer"]`;
        end
    end
    else
        Add $packet$ to `groups["datatransfer"]`;
    end
end
Initialize matrix $stats\_matrix \in \mathbb{R}^{5\times14}$ with zeros;
for $i = 0$ **to** 4 **do**
    $stats\_matrix[i, :] \leftarrow$ compute_stats($groups[i$-th group$]$);
end
Initialize matrix $protocol\_matrix \in \mathbb{R}^{32\times4}$ with zeros;
$protocol\_matrix \leftarrow$ extract_protocol_fields($packets[:32]$);
return $(stats\_matrix, protocol\_matrix)$;

Algorithm 3.1: Pcap processing flow

3.4.2. JA4+ Report

The replication of the *JA4+* experience report [17] was more straightforward than *MalDIST* replication. This was largely due to the authors' provision of a Github repository containing the dataset and the data preprocesing scripts [20] to look it up. So, basically this project takes the same exact dataset plus the Tria.ge Sandbox samples for benign traffic, translates and modifies some of the implementation of *JA4* and *JA4S* into and Python as it was originally in Perl format, include the implementation of *JA4X*, performs heatmaps and compare them to the article and develops a binary output machine learning model to classify traffic as benign or malware. Figure 3.4 shows a schematic representation of the *JA4+* approach implemented in this project:

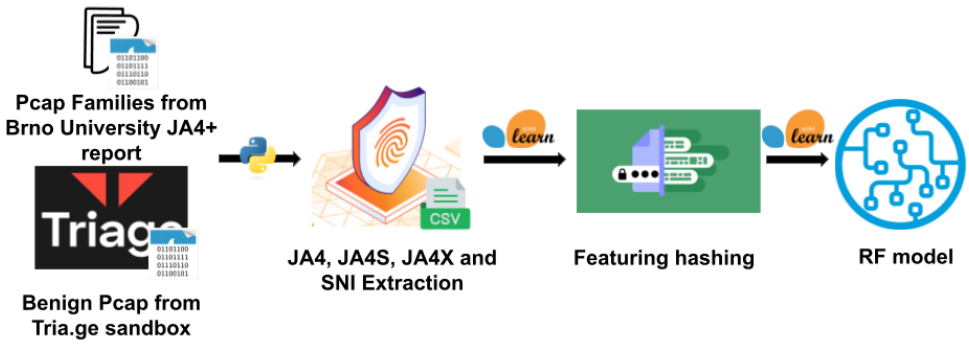


Figure 3.4: *JA4+* design

The steps to built up the *JA4*, *JA4S* and *JA4X* were the following:

- All extraction of the fingerprints from the PCAP files is performed in the script *get-ja4.sh* from [20], which will have some implementations in order to add *JA4X* fingerprints in the CSV file. The script has several parameters as arguments, but the ones used are:
  - **Input file**: PCAP file to extract characteristics
  - **App (-a)**: Appname to identify the family.
  - **Type (-t)**: Type of traffic: 0 for normal traffic, M for Malware and A for analytics.
  - **Outdir (-d)**: Directory where to safe the CSV.
  - **Whoisfile (-w)**: file which resolves IP addresses to domain names, essential to compute the *JA4* fingerprint domain variable.
- First thing the script does is to extract the features showed in table 3.1 with *tshark* and save it in a CSV named *pcap_file_name-extracted.csv*:

SrcIP	DstIP	TCP SrcPort	TCP DstPort	UDP SrcPort	UDP DstPort
Proto	type	Ciphersuite	List of Extensions	SNI	Supported Groups
EC	ALPN	Signature Algorithms	Supported Versions	Time	

Table 3.1: Tshark extracted feature to compute fingerprints

- With this CSV file with the extracted features, it is more straightforward to compute the fingerprints with no need of using any specialized library. Thus, with a Python script adapted from the Perl script [20], *JA4* and *JA4S* fingerprints are built and saved into a CSV file with the name *pcap_file_name-ja4.csv*.
- Finally, *JA4X* is computed with the Github repository of *JA4+* fingerprints of the original author [21] and will be joined to the CSV with the other fingerprints. The final CSV file will have {SrcIP, DstIP, SrcPort, DstPort, SNI, OrgName, JA4hash, AppName, Type, JA4Shash, JA4X, Filename, Version}.

After obtaining all the fingerprints for the entire dataset, the next step involves restructuring the data to create a balanced dataset for the binary output model. In this context, data augmentation is not feasible due to the categorical nature of the data.

Categorical data, in this context, refers to the *JA4+* fingerprints (JA4, JA4S, and SNI). These fingerprints are not continuous numerical values, but rather discrete values representing categories or classes. For example, each unique JA4hash value represents a specific configuration of the TLS *Client Hello*, and the SNI represents a specific server name. Unlike numerical data, where you can interpolate new values, categorical data points must be one of the existing categories. Therefore, 'new' fingerprint values cannot be created that are meaningful by simply interpolating between existing ones. Data augmentation techniques like SMOTE, which rely on creating new data points by interpolating between existing ones, are designed for numerical data, not categorical data.

The dataset used for this binary classification comprises desktop malware and application samples, with 6 308 samples for malware and 5 507 samples for benign applications. Given the categorical nature of the JA4+ fingerprints, Feature Hashing is employed.

Feature Hashing (also known as the hashing trick) is a technique used to convert categorical features into numerical representations (i.e., feature vectors). Instead of creating a dictionary that maps each unique categorical value to an index, Feature Hashing applies a hash function to the categorical value. The output of the hash function is then used as an index in a fixed-size vector.

In this project, Feature Hashing is applied to the JA4hash, JA4Shash, and SNI values. The results show an accuracy of 0.9825 when using Feature Hashing with JA4hash, JA4Shash, and SNI. A slightly lower accuracy of 0.9512 is achieved when using Feature Hashing with only JA4hash and JA4Shash, which is anyway considered because SNI may be encrypted in future versions of TLS.

3.5. Hybrid Model Design and Implementation

This section outlines the design and implementation of a hybrid deep learning model created to accurately classify network traffic as either malicious or benign. The model combines JA4+ fingerprints and MalDIST statistical features to take advantage of the strengths of both approaches. The final Dataset is explained in Subsection 3.3.3 and the configuration of each branch is the following:

- *MalDIST* branch is balanced using SMOTE with target class size of 10 000 samples, so both malware and benign will have 10 000 samples
- *JA4+* branch will have 6 308 malware samples and 5 507 benign samples and will be using FeatureHasher on *JA4* and *JA4S*.

3.5.1. Data Balancing

Before training the hybrid model, it was essential to address potential class imbalances within the dataset. To achieve this, the following approach was adopted to ensure an equal number of malware and benign samples:

1. Ensure both the JA4+ data and the MalDIST data are in a suitable format.
2. Determine the minimum number of samples between JA4+ data and MalDIST data.
3. Truncate both data with the minimum number of samples. This ensures that both datasets have the same size.
4. Truncate the labels to match the number of samples.

This process ensures that the model is not biased towards the class with more data. After balancing the data, the split into training and testing sets will have stratified sampling techniques to maintain the overall class distribution.

3.5.2. Model Architecture

The architecture of the hybrid neural network model is illustrated in Figure 3.5.

The model consists of two main branches: one processing JA4+ fingerprints and the other processing MalDIST statistics. Each branch includes a dense layer with 256 units, ReLU activation, and L2 regularization (0.01), followed by a dropout layer (0.4). The outputs of these two branches are then concatenated. The combined output is processed through a dense layer with 512 units and ReLU activation. To prevent overfitting, a dropout layer with a rate of 0.6 is applied afterwards. The final step

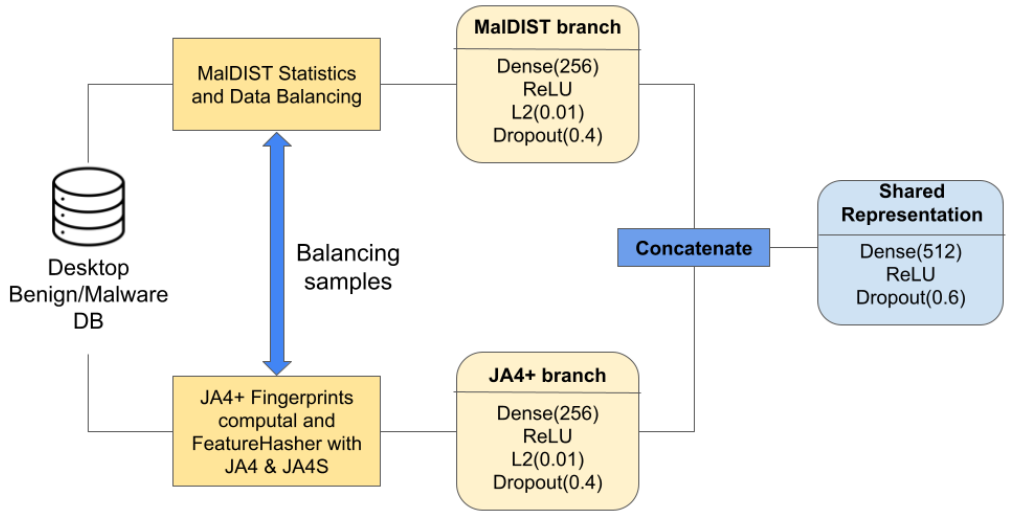


Figure 3.5: Model architecture

involves a dense output layer, which has as many units as there are classes, and uses a softmax activation function to generate the scores for each class.

3.6. Conclusions

This chapter has outlined the approach used and the array of tools employed to achieve the goals of this project. A large part of the effort was focused on designing and populating the databases, which form the backbone for the analysis. Importantly, marking successfully replication of prior research, specifically the MalDIST model, which not only confirmed existing findings but also set a benchmark for further improvements. By grouping malware families, *MalDIST* seemed to show enough information about the input file for its classification and distinction. In addition, the introduction of the JA4+ fingerprinting technique marked a key milestone. This method allowed us a meaningful extraction of features from the PCAP files, generating unique *JA4+* fingerprints, adding a valuable dimension to our analysis.

The *JA4+* fingerprinting method deserves particular attention. It provides a more detailed analysis of encrypted traffic by examining the specific characteristics of the *TLS* handshake. Compared to its predecessor, *JA3*, *JA4+* offers better accuracy and captures a wider array of identifiable characteristics, making it a strong tool to distinguish legitimate from malicious connections. The process of extracting these fingerprints from the PCAP files, which we detailed here, required careful attention to ensure that the data remained reliable and precise.

At the core of this project is the development of a hybrid model that combines two powerful techniques: statistical analysis of network flow data and insights derived from TLS fingerprints. Merging these approaches aims to deliver a more reliable and precise malware detection system, especially given

the complexities introduced by encrypted network traffic today. By using the strengths of each method and minimizing their weaknesses, this hybrid solution offers a more comprehensive approach. The next chapter will present the results achieved with this development.

4

Test and Validation

4.1. Introduction

This chapter walks through the testing and validation processes used to verify how precise the machine learning methods are. Provides a detailed investigation of the results of three key approaches: the *MalDIST* model, *JA4+* fingerprinting technique and a combined hybrid model that integrates both. In the statistical model, there is an evaluation of the models tested comparing the different metrics calculated (Accuracy, Precision, Recall and F1 Score) for the multiclass. The fingerprinting model first compares the unique fingerprints of the different malware families and the benign apps to see if there are some fingerprints contained in different families, and then it shows the Machine Learning approach results. Finally, there is a discussion of how the hybrid model could improve over the two setups, comparing results and efficiency. The chapter concludes with a summary of major findings and their significance for detecting malware in encrypted network traffic, emphasizing their potential impact on security practices.

4.2. MalDIST test and results

This section details the tests, results and analysis corresponding to *MalDIST* model. As a matter of fact, this article has provided some corrections and modifications in order to create a model more precise and realistic with the metric. Originally, *MalDIST* [2] consisted of a multimodal neural network combined which, in theory, was able to classify the same families as this model does, but it came with the following problems:

- **Complexity**: The model proposed 3 different branches of 128 nodes, one for payload, other for protocol field and the last one for the statistics. This architecture is overly complex for numerical features, which can be effectively handled by simpler models. This complexity could lead to memorization rather than generalization.
- **Payload**: Including the first 784 bytes of the first 32 packets of encrypted traffic for neural network

training introduces patterns which do not focus on any relevant flow property, as the encrypted traffic is unreadable, so the payload gives no meaningful information.

- **Data Balancing**: The original paper does not mention whether the samples for each class were balanced, nor the size of each class sample. If the samples were taken from the same sources as this project, which they did, the dataset would be unbalanced, with Hancitor having a disproportionately large number of samples, potentially leading to overfitting.

This section will present the results of the enhanced *MalDIST* proposal developed in this bachelor thesis, including both multi-class and binary class output.

4.2.1. MalDIST results

The new implementation leaves only the statistical matrix and the protocol matrix unified in a CSV file, where each flow has the 198 features. The initial test of the MalDIST model involves training it on a dataset containing several distinct malware families. This experiment aims to move beyond simple benign/malware classification and achieve a more granular categorization, identifying the specific malware family to which each network flow belongs. This multi-class classification provides more valuable insights into the nature of the malicious traffic.

To achieve this, several machine learning models were evaluated. The dataset was split into training and testing sets to assess the performance of each model. The following models were used:

- **Random Forest Classifier**: A robust ensemble learning method that constructs a multitude of decision trees at training time. It is known for its ability to handle high-dimensional data and provide good generalization. The project used a Random Forest Classifier with 50 estimators.
- **Extra Trees Classifier with Feature Selection**: Similar to Random Forest, Extra Trees builds multiple decision trees. However, it differs in how it chooses the best split. The project used Extra Trees Classifier with SelectFromModel for feature selection, in order to reduce the number of features and use only the most relevant ones, potentially improving the model performance and reducing complexity.
- **XGBoost Classifier**: XGBoost is an optimized distributed gradient boosting library designed to be highly efficient, flexible and portable. It implements machine learning algorithms under the Gradient Boosting framework.
- **K-Nearest Neighbors (K-NN)**: A non-parametic algorithm that classifies a data sample based on the majority class of its k nearest neighbors using k=10 in this case.
- **Logistic Regression**: A linear model for classification used for predicting categorical outcomes iterated a maximum of 1 000 times.

After contextualizing, the following metrics are obtained from the 5 different models:

Models	Metrics	*Results*
Random Forest Classifier	*Accuracy*	0.9904
	Precision	0.9905
	Recall	0.9904
	F1-score	0.9904
Extra Tree Classifier	*Accuracy*	0.9896
	Precision	0.9896
	Recall	0.9896
	F1-score	0.9896
XGBoost	*Accuracy*	0.9904
	Precision	0.9905
	Recall	0.9904
	F1-score	0.9904
K-NN	*Accuracy*	0.9606
	Precision	0.9605
	Recall	0.9605
	F1-score	0.9605
Logistic Regression	*Accuracy*	0.8274
	Precision	0.8284
	Recall	0.8270
	F1-score	0.8271

Table 4.1: Metrics of 3 MalDIST models

By looking at the results of table 4.1, it seems like the normal Random Forest is the optimal model. Although ExtraTree Classifier and XGBoost have similar results, Random Forest is simpler and it obtains the best results. The confusion matrix of the multiclass model with Random Forest is shown in figure 4.1, where 0 represents Benign, 1 Dridex, 2 Emotet, 3 Hancitor and 4 Valak.

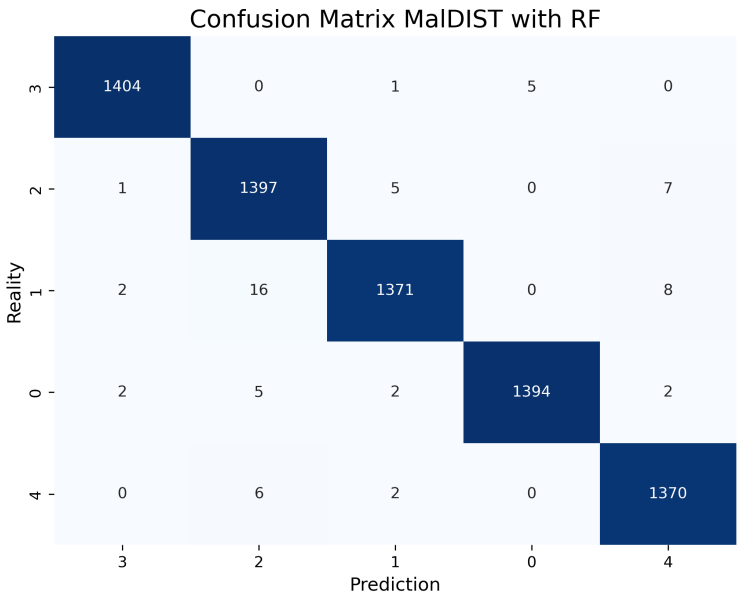


Figure 4.1: MalDIST multiclass Confusion Matrix

Random Forest will be chosen to perform the Binary Output (Benign or Malware) obtaining results of 0.9792 of accuracy.

4.3. JA4+ test and results

This section details the tests, results and analysis corresponding to *JA4+* experience report [17]. It implements and analyzes the same heatmaps proposed in that paper, but with a different approach and results.

Firstly, the fingerprints are grouped by the family, generating a dictionary where each family contains the unique fingerprints. This will be helpful to identify one fingerprint to a family and to know how similar are two families of malware, or benign with malware. Table 4.2 shows some visual statistics of *JA4+* of desktop apps and desktop malware and Table 4.3 a concrete example (Dridex):

	Total	Unique	Most Frequent Fingerprint	Frequency	Mean Frequency
JA4	9478	64	*t12d1909h2_d83cc789557e_7af1ed941c26*	2370	148.09
JA4S	7325	191	*t1205h2_c030_7136cef64a82*	1557	38.35
JA4X	5671	102	*a373a9f83c6b_2bab15409345_0f2217ba412e*	2684	55.60
JA4+JA4S	9549	345	*t12d1909h2_d83cc789557e_7af1ed941c26_t1205h2_c...*	1532	27.68
JA4+JA4S+JA4X	9549	603	*t12d1909h2_d83cc789557e_7af1ed941c26_t1205h2_c...*	1529	15.84

(a) Desktop Malware Stats

	Total	Unique	Most Frequent Fingerprint	Frequency	Mean Frequency
JA4	469	16	*t12d1909h2_d83cc789557e_7af1ed941c26*	255	29.31
JA4S	468	46	*t1206h2_c030_e1dda4771ae8*	66	10.17
JA4X	209	28	*a373a9f83c6b_2bab15409345_0f2217ba412e*	65	7.46
JA4+JA4S	471	69	*t12d1909h2_d83cc789557e_7af1ed941c26_t1206h2_c...*	66	6.83
JA4+JA4S+JA4X	471	95	*t13d1516h2_8daaf6152771_02713d6af862_t130200_1...*	39	4.96

(b) Desktop Apps Stats

Table 4.2: Statistics of *JA4+* fingeprints of the dataset

	Total	Unique	Most Frequent Fingerprint	Frequency	Mean Frequency
JA4	53	2	*t12d1909h2_d83cc789557e_7af1ed941c26*	52	26.50
JA4S	53	4	*t1205h2_c030_7136cef64a82*	42	13.25
JA4X	52	1	*a373a9f83c6b_2bab15409345_0f2217ba412e*	52	52.00
JA4+JA4S	53	2	*t12d1909h2_d83cc789557e_7af1ed941c26_t1205h2_c...*	42	13.25
JA4+JA4S+JA4X	53	2	*t12d1909h2_d83cc789557e_7af1ed941c26_t1205h2_c...*	42	13.25

Table 4.3: Statistics of *JA4+* fingeprints of Dridex

The *Dridex* example perfectly represents the aim of this distribution having few unique fingerprints with high frequency for the whole family. Thus, there is a fingerprint label to this family making it posible to identify it. Now the objective is to compare each malware family between them and the malware families between Benign Apps and see how many fingerprints they share.

In order to clearly compare the families, the heatmap approach of this project is a simmetrix matrix to compare Malware Families, where the rows and columns are the Malware Families, and a matrix for comparing bening apps with Malware Families, where the rows are the Malware Families and columns are the Benign Apps. Both heatmaps are grey scale heatmaps, where the darker the color is, the more common fingerprints they have. The operation to calculate the grey color is the following:

$$\text{Similarity} = \frac{|\text{SET(row)} \cap \text{SET(col)}|}{|\text{SET(row)}| + |\text{SET(col)}|} \tag{4.1}$$

where the row set contains the unique fingerprints of the row family and the column set contains the unique fingerprints of the column family. That is why the maximum number this equation can reach is 0.5, which will mean that the two families are completely equal, which is what happens in the symmetric matrix comparing malware families, as shown in figure 4.2.

Once this has been explained, the heatmaps created are organized in two sets: the first set contains the comparison between Malware Families with the different *JA4+* fingerprints and *SNI* and combining them; and the second set contains the comparison between Malware Families and Benign Apps with the same configuration.

a) **JA4 Heatmap**: Little distinctions between families can be seen, as the client is the same for all the samples it makes sense that they share fingerprints. However, it can be appreciated the exact behavior of Dridex and Emotet.

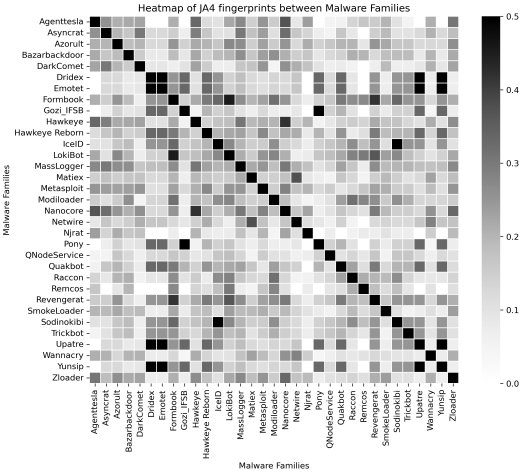
b) **JA4+JA4S Heatmap**: Adding *JA4S* facilitate to distinguish the families, making them more unique. Seems that the server's respond contrasted with the same client provides more differentiating information.

c) **JA4+JA4X Heatmap**: The inclussion of *JA4X* also changes the similarity patterns but it is not clear if it improves *JA4+JA4S* as it seems that the more grey squares are now lighter but the light squares now are darker. The only exception is *Gozi_IFSB* which is a peculiar case, as the only fingerprints it has are two with Bing as SNI and the exact fingerprint as some of older versions of Bing but with different *JA4X*.

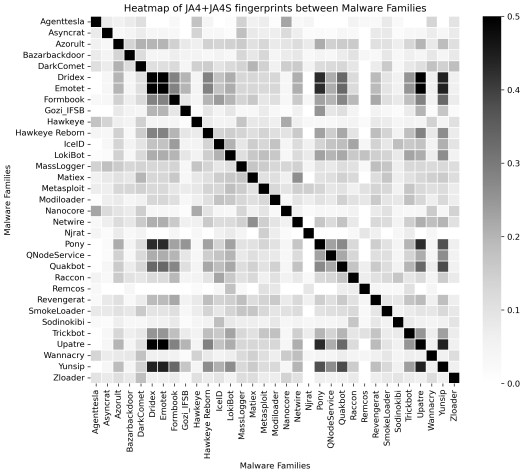
d) **JA4+SNI Heatmap**: Seems to be the option with most impact on relationships.

e) **JA4+JA4S+JA4X**: It improves the dual combinations it seems to be worse than the option with *SNI*

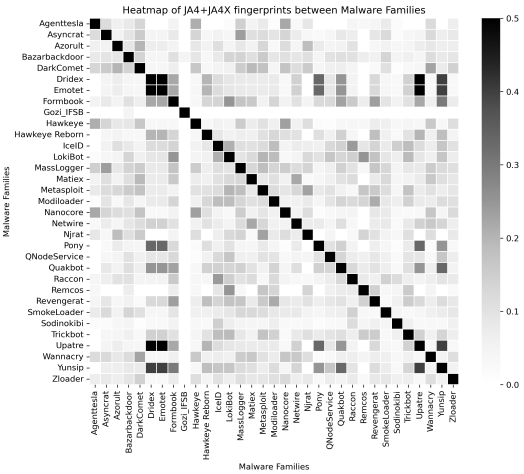
f) **JA4+JA4S+JA4X+SNI**: It barelly improves *JA4+SNI* option.



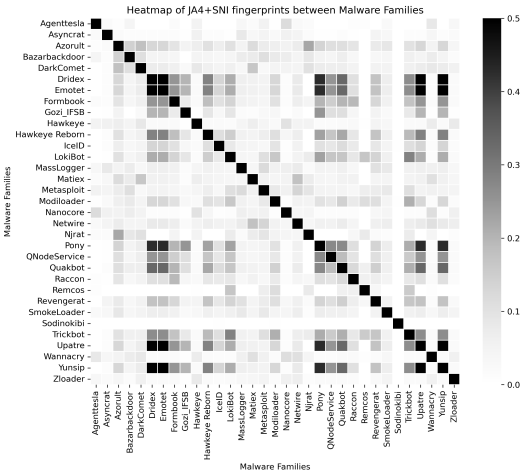
(a) Heatmap of JA4 fingerprints between Malware Families



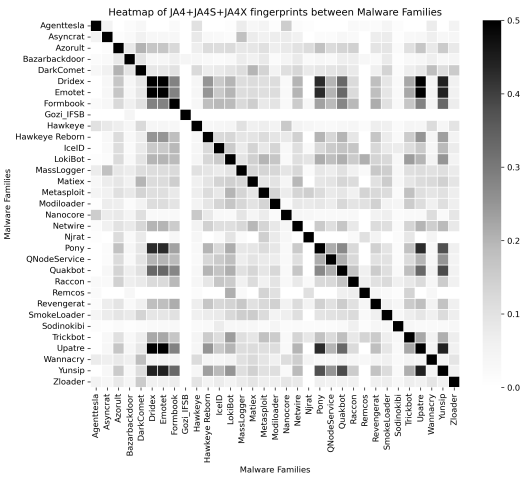
(b) Heatmap of JA4+JA4S fingerprints between Malware Families



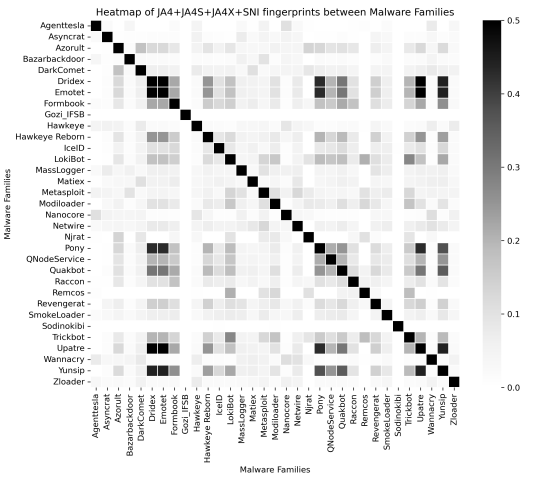
(c) Heatmap of JA4+JA4X fingerprints between Malware Families



(d) Heatmap of JA4+SNI fingerprints between Malware Families



(e) Heatmap of JA4+JA4S+JA4X fingerprints between Malware Families



(f) Heatmap of JA4+JA4S+JA4X+SNI fingerprints between Malware Families

Figure 4.2: Heatmaps comparison of *JA4+* figerprints of Malware Families

Figure 4.3 shows a clear difference between all the fingerprints from mobile and desktop either malware and apps. As it was mentioned in the previous set explanation *Gozi_IFSB* seems to be really similar to many benign apps, due to its Bing fingerprints. These similarities dissapear as long as the fingerprint are combined until the best option is reached: *JA4+JA4S+SNI*, thanks to *SNI* which adds server context having a complete blank matrix. This conclusion was also reached by [17], but with the other combinations there is also a solid solution to differentiate malware from benign apps.

4.3.1. Feature Hasher Model

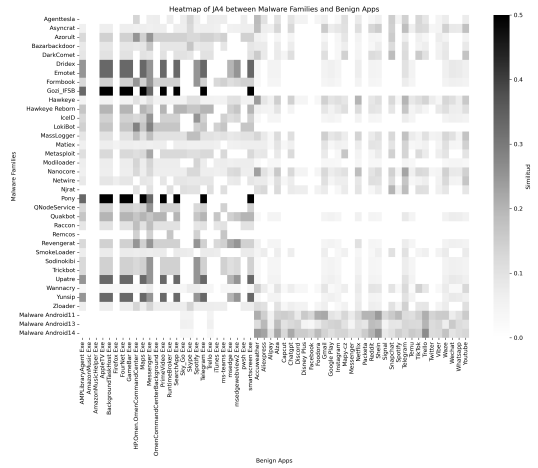
In this study, we extend prior work to classify malware and benign traffic. The *JA4+* fingerprint data, consisting of strings representing combinations of *JA4*, *JA4S*, *JA4X* and *SNI*, is transformed into a numerical format suitable for machine learning. Since fingerprints are categorical data, non-numeric, Machine Learning models are not able to compare the data unless this is transformed. One hot encoding, Levenshtein Distance are some options to compare categorical data, but due to the high dimensionality and variability it will be very complex. To address this, the *FeatureHasher* from *scikit-learn* library is employed.

The *FeatureHasher* works by applying a hashing function to the input strings (JA4 hashes). This function maps each unique string to a set of indices within a fixed-size feature vector. The key of *FeatureHasher* is that instead of creating a dictionary of all possible strings, it uses the hashing function to determine where each string contributes to the feature vector.

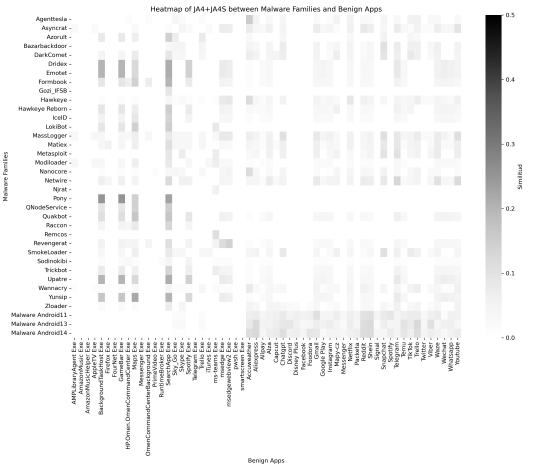
FeatureHasher has a number of features (n_features) which determines the dimensionality of the resulting numerical vector. In this implementation, *n_features* is set to 1024. This value was chosen due to the high variety of different *JA4+* fingerprints. A larger number of features allows the model to capture more distinctions between different fingerprints.

The resulting hashed feature vectors are then used to train a Random Forest Classifier. The performance of the model is evaluated using metrics such as precision, recall, and F1-score, as shown in table 4.4. Three models were tested, one transforming *JA4_JA4S*, another *JA4_JA4S_SNI* and the last one transforming *JA4_JA4S_JA4X*.

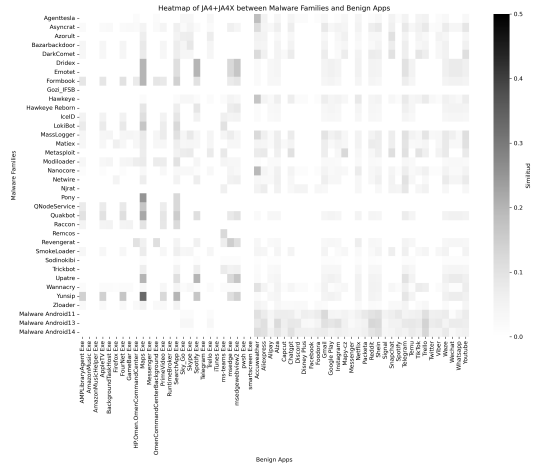
As observed in previous results, the best option is *JA4_JA4S_SNI*, with a minimum difference over the model with *JA4X*. The evolution of encrypted traffic over the network advances to show less information, making future *TLS* handshake to encrypt SNI and the certificate necessary to build *JA4X* fingerprint, which forced to make a solution based only on the client and server fingerprints, which still has good results in predictions.



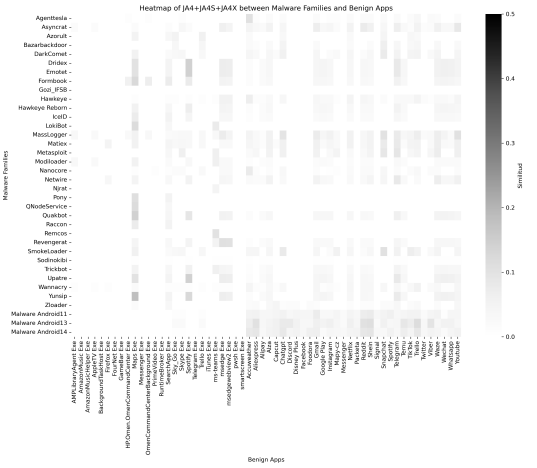
(a) Heatmap of JA4 fingerprints between Malware Families and Benign Apss



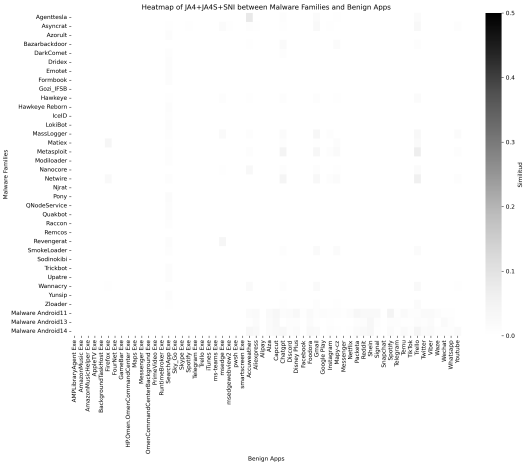
(b) Heatmap of JA4+JA4S fingerprints between Malware Families and Benign Apps



(c) Heatmap of JA4+JA4X fingerprints between Malware Families and Benign Apps



(d) Heatmap of JA4+JA4S+JA4X fingerprints between Malware Families and Benign Apps



(e) Heatmap of JA4+JA4S+SNI fingerprints between Malware Families and Benign Apps

Figure 4.3: Heatmaps comparison of *JA4+* figerprints of Malware Families vs Benign Apps

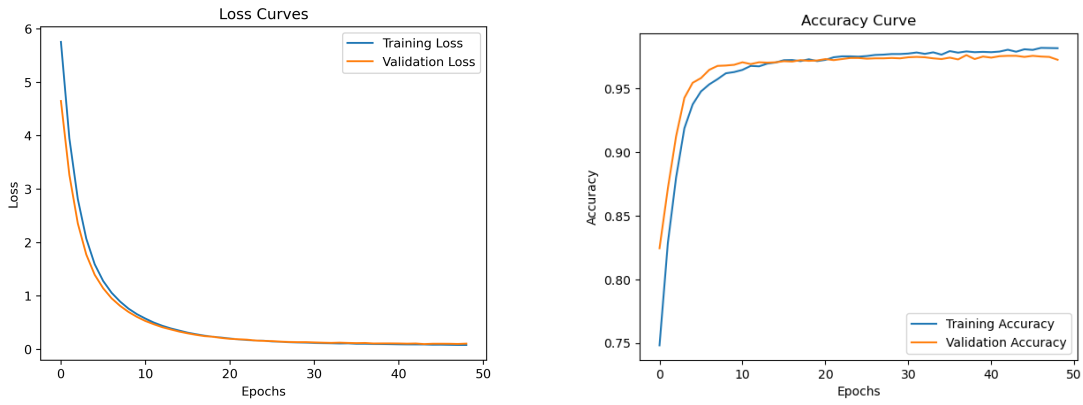
Models	Metrics	*Random Forest*
JA4_JA4S	*Accuracy*	0.9512
	Precision	0.9511
	Recall	0.9532
	F1-score	0.9511
JA4_JA4S_SNI	*Accuracy*	0.9825
	Precision	0.9822
	Recall	0.9827
	F1-score	0.9824
JA4_JA4S_JA4X	*Accuracy*	0.9724
	Precision	0.9717
	Recall	0.9733
	F1-score	0.9723

Table 4.4: Metrics of 3 FeatureHasher models

4.4. Hybrid model test and results

The testing phase of our final model follows a thorough evaluation process designed to ensure its effectiveness and dependability.

First, during the training, the use of an Adaptative Moment Estimation (Adam) adjusted to a slow learning rate to ensure stable and efficient convergence. To prevent overfitting and optimize the training process, early stopping is employed. This technique monitors the model performance on a validation set and stops the training process when the validation loss stops improving. The training process aims to reduce the gap between the model predictions and the actual labels, measured by a loss function. Figures 4.4(a) and 4.4(b) show the progressive error loss curve and the increase accuracy curve:



(a) Loss Curve

(b) Accuracy curve

Figure 4.4: Hybrid Model training

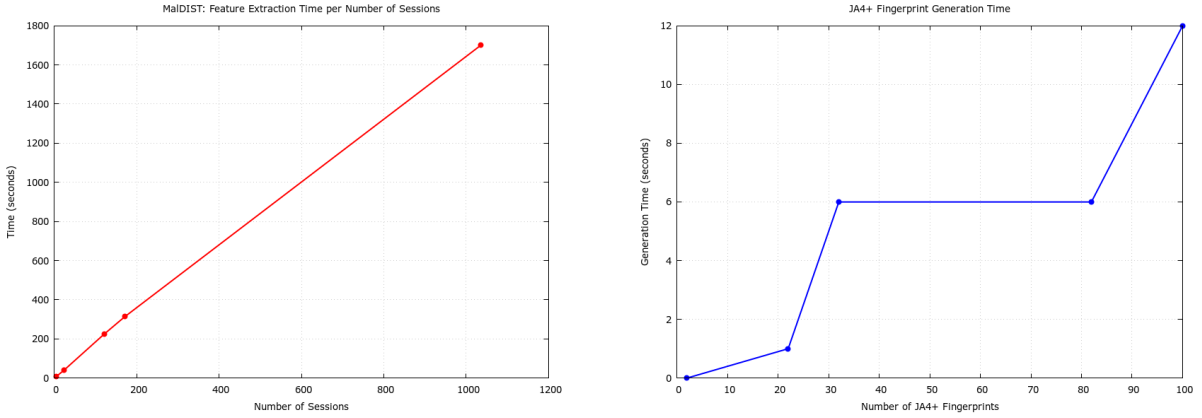
The hybrid model achieves strong performance, with an accuracy of 0.9757, a precision of 0.9755, a recall of 0.9758, and an F1 score of 0.9756. These results are comparable to those of the individual models from which this multimodal neural network is constructed, suggesting that the increased complexity of the hybrid model does not significantly degrade accuracy performance. However, it also

suggests that simpler models can work faster, which is important to cut an attack promptly.

4.4.1. Time measurements

In the context of prediction models and classifying classes, the time of preprocesing the data is crucial to assess the model efficiency. In Malware Detection, time is even more critical in order to do real-time analysis and threat mitigation. This section will evaluate the time taken for processing and classifying network traffic using the different models.

To achieve this goal, some PCAP files for both *MalDIST* and *JA4+* models have been processed, measuring the time. Figure 4.5 shows two graphics of time spent per size of the sample. The samples taken in order are: *SnakeKeylogger*, *Dridex Infection Traffic*, *Emotet Quackbot*, *Whatsapp Exe* and *Emotet Epoch4 spambot*.



(a) *MalDIST* flow separation and feature extraction time per number of flows

(b) *JA4+* fingerprints construction time per number of fingerprints

Figure 4.5: Time Execution of Processing model's data

Figure 4.5 shows a high time difference between the two models, being the fingerprinting by far more efficient. This is because *MalDIST* has to go through two phases: first separating the PCAP file into different flows, and then extract the features of each flow. Individually extracting features of one flow is not highly cost, but the amount of flows makes the model heavier. By contrast, *JA4+* just looks at the *TLS* handshake and extract the corresponding characteristics to build up its fingerprints.

The hybrid model has the two preprocessing methods as inputs, which will make the fingerprinting preprocessing to wait for the statistical features to be ready in the majority of cases. This introduces an additional layer of complexity and potential delay, as the two processes are dependent on each other. Further analysis will be needed to optimize this interaction and minimize the overall processing time of the hybrid model.

4.5. Conclusions

This chapter has explored different results of the two models studied, plus the efficiency of the predicting models. It has analyzed the previous work weaknesses and how to improve them, and also review the results and compared them with the proposed ones. It presents some statistics about the fingerprints which helped to perform the heatmaps. Summarizes the prediction models applied and explored the efficacy of combining *JA4+* fingerprints and statistical flow analysis for detecting malware in encrypted network traffic.

In summary, the testing and validation phase involved a comparative analysis of the three machine learning models, all configured with a binary output, meaning that they predict if a network flow contains malicious traffic or just normal. While all three models demonstrated comparable performance metrics, the *JA4+* model distinguishes itself primarily in terms of processing efficiency. The *JA4+* model achieves this efficiency in time processing, focusing only on the calculation of fingerprints from the initial packet of each network session, whereas *MalDIST* needs to process many PCAP files, each one representing an individual flow. This approach minimizes the computational overhead associated with processing entire PCAP files, offering a substantial advantage in real-time malware detection scenarios. The reduced processing time, coupled with the competitive accuracy of JA4+ makes it a compelling choice for practical deployment.

5 Conclusions and Future Work

5.1. Conclusions

This bachelor thesis explored the application of machine learning techniques to address the challenge of malware detection in encrypted network traffic with a particular focus on *JA4+* fingerprinting and statistical flow analysis. Taking into account all the information presented through the document, the most important part of the information covered in each section is the following:

- **State of Art**: The analysis of the limitations of traditional approaches, combined with the complete explanation of methodologies of new *TLS* fingerprinting tools set a strong foundation for the research presented in this bachelor thesis.
- **Design and Implementation**: This work addressed the challenge of malware detection by creating a comprehensive dataset. This was achieved by combining data from various sources and generating new samples of benign traffic using the *Tria.ge* Sandbox tool. Additionally, three malware detection approaches were implemented and compared: the replication of an existing model, *MalDIST*, a new model based on *JA4+* fingerprints and a new hybrid model that combines statistical traffic features with the fingerprints.
- **Test and Validation**: The performance of the implemented models was evaluated using standard evaluation metrics. The results demonstrated the effectiveness of the *JA4+*-based approach for detecting malware in encrypted traffic, achieving performance comparable to the *MalDIST* model with significantly better computational efficiency. Specifically, the *JA4+* model showed the ability to effectively detect malware while requiring fewer computational resources.

5.2. Future Work

It is difficult to tell how to focus future work, as the network traffic is in continuous change, encrypting more and more information to protect the users' sensitive data. Malware can benefit from this encryption as well, camouflaging their malicious intentions and infecting the users' computer. Anyways,

the collection of two different datasets from different sources and the creation of new benign samples using *Tria.ge* Sandbox makes a complete dataset to be tested with the novel model as it is the *JA4+* fingerprints tool. The possible future investigation could be:

- **Enhanced Dataset and Model Refinement**: Collect more Benign samples from other resources and train and test a new *JA4+* model with this Benign samples and more updated and more quantity Malware samples from *malware-traffic-analysis* [18]. The problem of the model was the limitation of malware samples with the fewer amount of *JA4+* fingerprints generated by Benign samples.
- **Efficient Statistical Modeling**: Further research could explore methods for developing more efficient statistical models. Specifically, investigating techniques to reduce the computational overhead associated with processing large volumes of network traffic data, while retaining or improving model accuracy, would be valuable.
- **Real-time Malware Detection System**: A significant area for future work is the development of a real-time malware detection system. This would involve designing a system capable of processing network traffic streams and providing timely predictions of whether the traffic is malicious or benign, or in addition, predict the malware family or type of benign app.

Bibliography

[1] F5, "Detect Encrypted Malware with SSL Inspection." [Online] Available: https://www.f5.com/resources/library/encrypted-threats/ssl-visibility/encrypted-malware [Accessed 10-april-2023].

[2] O. Bader, A. Lichy, C. Hajaj, R. Dubin, and A. Dvir, "Maldist: From encrypted traffic classification tomalware traffic detection and classification," *IEEE*, pp. 527–533, 2022.

[3] Zscaler, "ThreatLabz 2023 State of Encrypted Attacks Report." [Online] Available: https://www.zscaler.com/resources/2023-threatlabz-state-of-encrypted-attacks-report [Accessed 12-april-2023].

[4] Kurt Baker, "The 12 Most Common Types of Malware." [Online] Available: https://www.crowdstrike.com/en-us/cybersecurity-101/malware/types-of-malware/ Accesed: 26-april-2025.

[5] E. Rescorla, "The Transport Layer Security (TLS) Protocol Version 1.3," *RFC*, vol. 8446, pp. 1–160, 2018.

[6] Fastly, "What is TLS Fingerprinting." [Online] Available: https://www.fastly.com/blog/the-state-of-tls-fingerprinting-whats-working-what-isnt-and-whats-next [Accesed: 23-april-2025].

[7] John Althouse, "JA4+ Network Fingerprinting," *FoxIO*, 2023. [Online] Available: https://medium.com/foxio/ja4-network-fingerprinting-9376fe9ca637 [Accesed: 23-april-2025].

[8] John Althouse, "TLS Fingerprinting with JA3 and JA3S." [Online] Available: https://engineering.salesforce.com/tls-fingerprinting-with-ja3-and-ja3s-247362855967/ [Accesed: 23-april-2025].

[9] D. Cooper and S. Santesson, "Internet X.509 Public Key Infrastructure Certificate and Certificate Revocation List (CRL) Profile," *IETF RFC 5280*, 2008. [Online] Available: https://datatracker.ietf.org/doc/html/rfc5280 [Accessed 24-april-2023].

[10] L. Nataraaj, "Malware images: Visualization and automatic classification," *ACM*, 2011.

[11] W. Wang, M. Zhu, J. Wang, X. Zeng, and Z. Yang, "Malware traffic classification using convolutional neural network for representation learning," *IEEE*, pp. 712–717, 2017.

[12] W. Wang, M. Zhu, J. Wang, X. Zeng, and Z. Yang, "End-to-end encrypted traffic classification with one-dimensional convolution neural networks," *IEEE*, pp. 43—-48, 2017.

[13] T. Shapira and Y. Shavitt, "Flowpic: A generic representation for encrypted traffic classification and applications identification," *IEEE*, pp. 680–687, 2021.

[14] G. Aceto, D. Ciuonzo, A. Montieri, and A. Pescape, "DISTILLER: encrypted traffic classification via multimodal multitask deep learning," *Science Direct*, vol. 183–184, pp. 1–19, 2021.

[15] O. Bader, A. Lichy, C. Hajaj, R. Dubin, and A. Dvir, "Open-Source Framework for Encrypted Internet and Malicious Traffic Classification," *arxiv*, pp. 1–11, 2022.

[16] B. Anderson and D. McGrew, “Accurate TLS fingerprinting using destination context and knowledge bases,” *arXiv preprint arXiv:2009.01939*, pp. 1–14, 2020.

[17] P. Matousek, O. Rysavy, and I. Burgetova, “Experience Report: Using JA4+ Fingerprints for Malware Detection in Encrypted Traffic,” in *2024 20th International Conference on Network and Service Management (CNSM)*, pp. 1–5, IEEE, 2024.

[18] “Malware-traffic-analysis.net.” [Online] Available: https://www.malware-traffic-analysis.net/index.html [Accessed: 28-april-2025].

[19] Canadian Institue for Cybersecurity, “VPN-nonVPN dataset (ISCXVPN2016).” [Online] Available: https://www.unb.ca/cic/datasets/vpn.html [Accessed: 28-april-2025].

[20] matousp, “malware-analysis.” [Online] Available: https://github.com/matousp/malware-analysis/tree/main [Accessed: 1-may-2025].

[21] FoxIO-LLC, “ja4.” [Online] Available: https://github.com/FoxIO-LLC/ja4 [Accessed: 1-may-2025].


UÁM
Universidad Autónoma
de Madrid